

## Kopiec (heap)

Struktura danych stosowana w algorytmach kolejki priorytetowej, sortowania, statystyk pozycyjnych.

### Def. 1.

Kopiec to dowolne drzewo, w którym dla każdej pary wierzchołków postaci  $(v, \text{parent}(v))$  zachodzi warunek kopca:

$$\text{key}(\text{parent}(v)) \geq \text{key}(v).$$

(jest to wersja "max-kopiec", w wersji "min-kopiec" dla każdej takiej pary nierówność jest odwrotna.)

czyli:

- na dowolnej ścieżce od korzenia w dół wartości klucza się zmniejszają,
- zatem w korzeniu jest element największy,
- zaś najmniejszy element jest w którymś liściu.

### Def. 2.

Kopiec binarny (również: kopiec binarny zupełny) to drzewo binarne, w którym

- dla każdego wężła zachodzi warunek kopca,
- wszystkie poziomy, za wyjątkiem być może ostatniego, są całkowicie wypełnione,
- ostatni poziom jest wypełniony od lewej strony.

### Własności kopca binarnego:

$n$  = liczba wierzchołków,  $h$  = wysokość .

$h(n) = ?$

- na głębokości  $d, d=0, \dots, h-1$ :  $2^d$  węzłów;
- na głębokości  $h$ :

$$n - (1+2+4+\dots+2^{h-1}) = n - 2^h + 1$$

co najmniej 1, co najwyżej  $2^h$  węzłów;

zatem:

$$1 \leq n - 2^h + 1 \leq 2^h, \quad // \text{dodać stronami } 2^h$$

$$2^h \leq n < 2^{h+1},$$

$$h \leq \lg n < h+1.$$

$h$  – całkowite, więc:

$$h = \lfloor \lg n \rfloor.$$

### **Realizacja kopca w tablicy:**

- tablica  $a[1..size]$ ,  $size \geq n$ ,
- elementy kopca w  $a[1..n]$ ,
- kolejno poziomami, tj.:  $a[1]$  = korzeń,
- $a[2..3]$  = węzły na głębokości 1,
- $a[4..7]$  = węzły na głębokości 2, itd.

### **Fakt 1:**

Jeśli wierzchołki drzewa ulokowano w tablicy jak wyżej, to dla każdego  $i=1, \dots, n$ , prawym następnikiem wierzchołka  $a[i]$ , o ile istnieje, jest wierzchołek  $a[2i+1]$ .

### **Dowód:**

Dla każdego drzewa binarnego mamy:

$$n_0 = n_2 + 1,$$

gdzie  $n_d$  oznacza liczbę wierzchołków stopnia  $d$ .

Stąd, jeśli każdy wierzchołek ma stopień 2 lub 0, to

$$(*) \quad n = n_0 + n_2 = 2n_2 + 1.$$

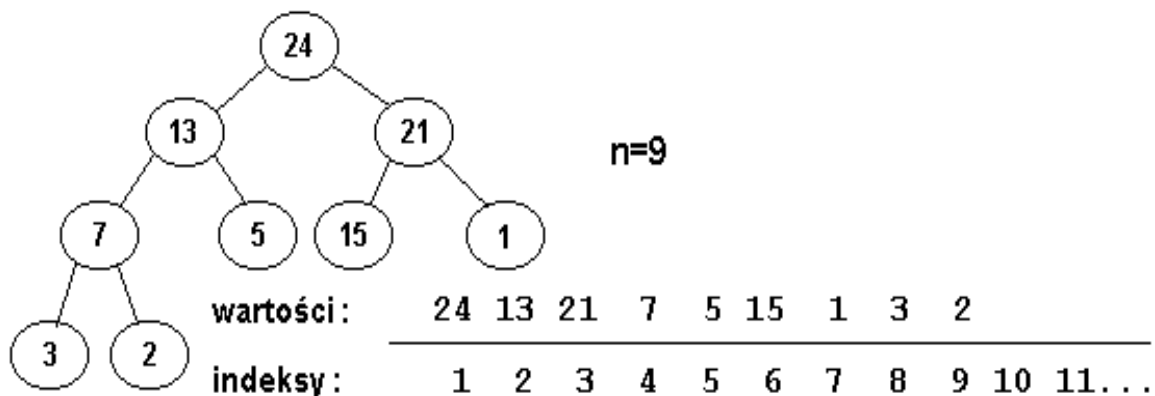
Niech  $a[j]$  - prawy następnik  $a[i]$ . Wykażemy  $j=2i+1$ .

- $a[1..j]$  jest początkowym fragmentem kopca, a więc też (mniejszym) kopcem; jego moc to  $n=j$ ;
- $a[j]$  jest ostatnim liściem w tym mniejszym kopcu,
- zatem  $a[i]$  jest ostatnim nie-liściem, wierzchołkiem stopnia 2, zatem  $n_2 = i$ ,
- a więc  $n = j = 2i + 1$ . QED

### Wniosek:

- lewym następnikiem wierzchołka  $a[i]$ , o ile istnieje, jest wierzchołek  $a[2i]$ ;
- poprzednikiem węzła  $a[i]$ , o ile istnieje, jest wierzchołek  $a[\lfloor i/2 \rfloor]$ .

Zatem przesuwanie się w dół i w górę kopca jest bardzo łatwe.



### Uwaga:

przy indeksowaniu tablic od zera, analogiczne wzory dla tablicy  $a[0..n-1]$  mają postać:

- następniki  $a[i]$  to wierzchołki  $a[2i+1]$ ,  $a[2i+2]$ ;
- poprzednik  $a[i]$  to wierzchołek  $a[\lfloor (i-1)/2 \rfloor]$ .

## Przesiewanie w górę:

- zakładając, że wartość  $a[k]$  zwiększyła się, przywróć warunek kopca na ścieżce od  $a[k]$  do korzenia;
- w tym celu zamieniaj kolejne pary (poprzednik, następnik), aż trafisz na poprawny porządek w parze, albo dojdiesz do korzenia;
- technicznie, poniższa wersja nie wykonuje zamian, lecz przesuwą na ścieżce elementy mniejsze niż  $a[k]$  o jedną pozycję w dół i następnie wstawia element  $a[k]$  na miejsce pozostawione przez ostatni przesuwany w dół element (podobnie jak w insertionsort).

```
UpHeap(a, k);  
// jak podczas sortowania listy przez wstawianie  
// listą jest ścieżka od wierzchołka k do korzenia  
j := k;  
i :=  $\lfloor k/2 \rfloor$ ;  
tmp := a[k];  
while j > 1 and a[i] < tmp do  
    a[j] := a[i];  
    j := i;  
    i :=  $\lfloor i/2 \rfloor$ ; // przejdź do rodzica  
// teraz element a[k] na swoje miejsce  
a[j] := tmp;  
end.
```

## Złożoność:

- dominują porównania elementów kopca,
- pesymistycznie:  $\lfloor \lg k \rfloor$ ,
- zatem:  $\Theta(\log n)$ .

## Przesiewanie w dół:

zakładając, że wartość  $a[k]$  zmniejszyła się, przywróć warunek kopca w poddrzewie o korzeniu  $a[k]$ .

```
DownHeap (a, k, n);  
// jak podczas sortowania listy przez wstawianie  
// lista (ścieżka do liścia) wyznaczana dynamicznie  
    tmp := a[k];  
    while k <= ⌊n/2⌋ do  
        j := 2*k;  
        if j < n and a[j] < a[j+1] then j++;  
        // a[j] - większy z następników węzła a[k]  
        if tmp >= a[j] break;    // warunek kopca ok.  
        // wpp. przesun aktualny element do góry  
        a[k] := a[j];  
        k := j;  
    // teraz element a[k] na swoje miejsce  
    a[k] := tmp;  
end.
```

## Złożoność:

Na każdym poziomie dwa porównania:

- wybór większego z następników,
- oraz porównanie z przesiewanym elementem.

Pesymistyczna liczba porównań:

- $2 * \text{różnica poziomów węzła } k \text{ i liści.}$
- dokładniej:  $2 * \lfloor \lg(n/k) \rfloor$

Najgorszy przypadek:

- $k=1$  (korzeń),
- przesiewanie do ostatniego poziomu;

zatem złożoność:  $\Theta(\lg n)$ .

## Kolejka priorytetowa

to struktura danych z operacjami:

- $\text{Insert}(Q, e)$ : wstaw element  $e$  do kolejki  $Q$ ,
- $\text{Max}(Q)$ : podaj wartość największego elementu w kolejce  $Q$ ,
- $\text{DeleteMax}(Q)$ : usuń największy element z kolejki,

ewentualnie dodatkowo:

- $\text{Construct}(Q, a_1, \dots, a_n)$ : utwórz kolejkę z podanych elementów.

Kopiec jako kolejka priorytetowa:

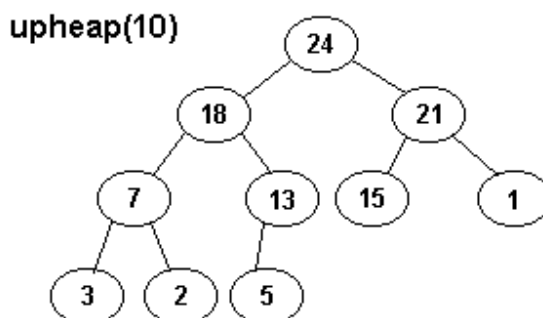
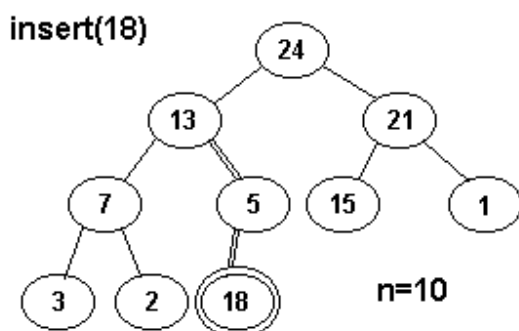
- tablica  $a[1..\text{maxsize}]$ , kopiec na miejscach  $a[1..n]$ .

```
Max()                                // odczyt największego
    return a[1];
end.
```

Złożoność:  $\Theta(1)$

```
Insert(e)                            // wstawianie elementu
// dopisz nowy liść na końcu i przesiej w górę
    a[++n] := e;
    UpHeap(a, n);
end.
```

Złożoność:  $\Theta(\lg n)$



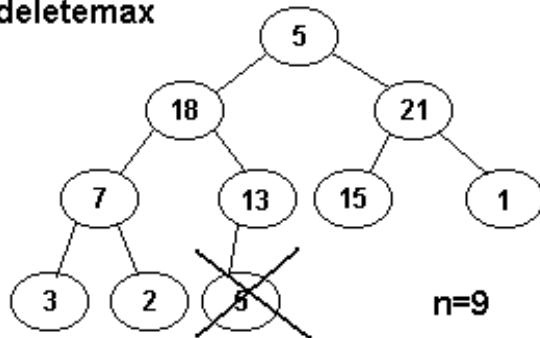
```

DeleteMax()                // usuwanie największego
// wstaw ostatni liść w miejsce korzenia,
// skróć kopiec o 1, i przesiej korzeń w dół
    a[1] := a[n--];
    DownHeap (a, 1, n);
end.

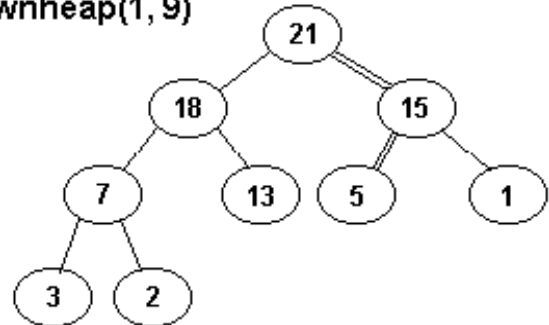
```

Złożoność:  $\Theta(\lg n)$

deletemax



downheap(1, 9)



## Sortowanie za pomocą kolejki priorytetowej

```

Create(Q, empty);
for i:=1 to n do
    Insert(Q, a[i]);
for i:=n downto 1 do
    a[i] := Max(Q);
    DeleteMax(Q);

```

- najpierw wszystkie elementy wstawiane do kolejki;
- następnie elementy opuszczają kolejkę w kolejności malejącej i wstawiane są do tablicy od końca;
- jeśli kolejka realizowana za pomocą kopca, to otrzymujemy sortowanie o złożoności  $\Theta(n \lg n)$ ;
- wada: kolejka-kopiec to dodatkowa tablica, oprócz sortowanej tablicy a[];

## Sortowanie metodą kopca (heapsort)

- wykorzystuje kopiec jako kolejkę priorytetową;
- kolejkę tę utrzymuje w sortowanej tablicy,

```
Heapsort (a, n);  
// faza 1: tworzenie kopca w tablicy a[];  
// jest to efektywna realizacja operacji Construct()  
// nie wykorzystująca operacji Insert()  
    for k :=  $\lfloor n/2 \rfloor$  downto 1 do  
        DownHeap(a, k, n);  
// faza 2: usuwanie z kopca  
    while n > 1 do  
        swap(a[1], a[n]);  
        n--;  
        DownHeap(a, 1, n);  
end.
```

### Poprawność fazy 1:

- (\*) założmy że w obu poddrzewach węzła  $a[k]$  warunek kopca jest wszędzie spełniony;
- pojedyncze wywołanie  $\text{DownHeap}(a, k, n)$  sprawia, że warunek kopca jest spełniony w całym poddrzewie o korzeniu  $a[k]$ ;
- jeśli wywołania  $\text{DownHeap}(a, k, n)$  powtarzamy poziomami, od liści do korzenia, to przed kolejnym wywołaniem zawsze spełniony jest warunek (\*);
- dla liścia  $a[k]$  nie ma sensu wywoływać  $\text{DownHeap}(a, k, n)$ ;
- węzeł o największym indeksie nie będący liściem to  $a[\lfloor n/2 \rfloor]$ , więc od niego zaczyna się konstruowanie kopca.

### Poprawność fazy 2: oczywista

Przykład:

indeksy elementów tablicy a:

1 2 3 4 5 6 7 8 9 10 11 12

wartości elementów:

10, 8, 4, 15, 13, 2, 18, 17, 6, 9, 3, 14

downheap (a, 6, 12):

10, 8, 4, 15, 13, 14, 18, 17, 6, 9, 3, 2

downheap (a, 5, 12): bez zmian

downheap (a, 4, 12):

10, 8, 4, 17, 13, 14, 18, 15, 6, 9, 3, 2

downheap (a, 3, 12):

10, 8, 18, 17, 13, 14, 4, 15, 6, 9, 3, 2

downheap (a, 2, 12):

10, 17, 18, 15, 13, 14, 4, 8, 6, 9, 3, 2

downheap (a, 1, 12):

18, 17, 14, 15, 13, 10, 4, 8, 6, 9, 3, 2

kolejne deletemax: zamień pierwszy z ostatnim

i zmniejsz granicę kopca o 1

2, 17, 14, 15, 13, 10, 4, 8, 6, 9, 3, | 18

i przesiej od korzenia w dół:

17, 15, 14, 8, 13, 10, 4, 2, 6, 9, 3, | 18

powtarzaj ten krok w sumie n-2 razy:

15, 13, 14, 8, 9, 10, 4, 2, 6, 3, | 17, 18

14, 13, 10, 8, 9, 3, 4, 2, 6, | 15, 17, 18

itd.

## Złożoność :

**Faza 1** - tworzenie kopca:  $T_1(n) = \Theta(n)$

Dowód:

- jeśli węzeł v jest na i-tym poziomie (licząc od dołu), to DownHeap(v) wykonuje  $\approx 2i$  porównań;
- na i-tym poziomie jest  $\approx n/2^i$  węzłów,  $i=1, \dots, \lg n$ ;
- razem porównań jest (poziom  $i=0$  pomijamy):

$$\sum_{i=1..lg n} 2ni/2^i \leq 2n \sum_{i=1..\infty} i/2^i = 2n.$$

$\sum_{i \geq 1} i/2^i = 2$ . Dowód:

$$\begin{aligned} S &= \sum_{i \geq 1} i/2^i = \sum_{i \geq 1} 1/2^i + \sum_{i \geq 1} (i-1)/2^i = \\ &= 1 + (1/2) \sum_{i \geq 1} (i-1)/2^{i-1} = 1 + (1/2) \sum_{i \geq 0} i/2^i = \\ &= 1 + (1/2) \sum_{i \geq 1} i/2^i = 1 + (1/2) S \Rightarrow \sum_{i \geq 1} i/2^i = 2 \end{aligned}$$

Inny dowód:

$$\begin{aligned} \sum_{i \geq 1} i/2^i &= \sum_{i \geq 1} \sum_{j=1..i} 1/2^i = \\ &\quad \begin{array}{ccc} 1/2 & & \text{sumę po wierszach zamień} \\ 1/4 & 1/4 & \text{na sumę po kolumnach:} \\ 1/8 & 1/8 & 1/8 \quad \dots\dots \end{array} \\ &= \sum_{j \geq 1} \sum_{i \geq j} 1/2^i = \sum_{j \geq 1} 1/2^{j-1} = 2 \sum_{j \geq 1} 1/2^j = 2 \end{aligned}$$

**Faza 2** - usuwanie wszystkich elementów z kopca:

- $n$  x DeleteMax, kosztuje  $T_1(n) = O(n \log n)$ ,
- można skonstruować dane, dla których faza 2 kosztuje  $\Theta(n \log n)$  (Fakt poniżej);
- to że faza 2 ma złożoność  $\Omega(n \log n)$ , w kontekście liniowej złożoności fazy 1, wynika również z dolnego ograniczenia  $\Omega(\log n!)$  na złożoność algorytmów sortujących przez porównania (omówione dalej).

Zatem złożoność Heapsort-u:  $T(n) = \Theta(n \log n)$ .

Decyduje złożoność fazy 2.

### **Fakt:**

Dla dowolnego  $n$  można skonstruować taki kopiec wielkości  $n$ , że w fazie 2 w każdym wywołaniu DownHeap() przesiewanie będzie sięgało liścia.  
(ćwiczenie).

Można obliczyć, że dla takich danych liczba porównań wykonanych w fazie 2 wynosi:

$$2n \lg n - f(n), \quad \text{gdzie } f(n) = \Theta(n).$$

**Fakt:**

- algorytm Heapsort w przypadku pesymistycznym wykonuje liczbę porównań bliską  $2n \lg n$ ,
- w przypadku średnim liczba wykonanych porównań jest również bliska  $2n \lg n$  (nieco mniejsza niż w przypadku pesymistycznym).

**Zalety sortowania metodą kopca**

(w stosunku do quicksortu):

- pesymistyczny czas tylko  $\Theta(n \lg n)$ ,
- nie potrzebuje pamięci roboczej (nie ma stosu rekurencji),
- bardzo prosty w implementacji.

**Wada:**

- średni czas jest prawie równy pesymistycznemu, więc jest wyraźnie dłuższy niż w quicksortcie, szczególnie dla dużych danych.

Podobnie jak quicksort, **heapsort jest niestabilny.**

**Uwaga:** powyższa, klasyczna wersja algorytmu Heapsort pochodzi od **Williamsa**.

**Wersja 2 (Floyd):**

Zmniejsza średnią liczbę porównań około dwukrotnie,

W Fazie 2 zmodyfikowana wersja DownHeap:

- przesuwa w górę całą ścieżkę od korzenia do liścia (większy z dwojga dzieci przesuwany w miejsce rodzica; jedno porównanie dzieci na każdym poziomie wystarcza do wyznaczenia ścieżki),

- wracając od liścia do góry elementy mniejsze od przesiewanego z powrotem zsuwamy w dół, wykonując jedno porównanie na każdym poziomie (tak jak w UpHeap),
- można dowieść, że średnia liczba porównań pojedynczej DownHeap wynosi  $\lg n + \text{const.}$ ;
- pesymistyczna liczba porównań pozostaje taka sama:  $2 \lg n$ .

## **Zastosowanie kopca do statystyk porządkowych**

### **Znajdowanie k-tego elementu:**

We:  $S$  - zbiór,  $k \leq |S|$ ;

Wy:  $k$ -ty co do wielkości element zbioru  $S$  (licząc od najmniejszego).

Metoda zalecana dla  $k$  bliskich 1 lub  $n$ .

Np. dla  $k$  bliskiego 1:

- utworzyć  $H = \text{min-kopiec}(S)$ ;
- wykonać  $k-1$  razy  $\text{DeleteMin}(H)$ ;
- odczytać korzeń – jest to  $k$ -ty element.

Złożoność:

- tworzenie kopca:  $\Theta(n)$
- $(k-1)$  razy  $\text{Deletemin}$ :  $\Theta(k \lg n)$
- razem:

$$\Theta(n + k \lg n)$$

Dla  $k$  bliskiego  $n$  tworzymy max-kopiec i wykonujemy  $n-k$  razy  $\text{Deletemax}$ .

## **Jak szybko można sortować ?**

### **Dolne ograniczenia na złożoność sortowania**

Czy istnieją algorytmy sortujące w czasie  $O(n \log n)$  ?  
("o małe").

Konieczna formalna specyfikacja modelu obliczeń:  
**drzewo decyzyjne.**

Opisywalne w nim są algorytmy sortowania, w których:

- typ elementów jest nieistotny,
- jedyne dopuszczalne operacje na elementach sortowanych to wzajemne porównania,
- porównanie dokonuje "czarna skrzynka" w czasie równym 1.

Mieszczą się w tym modelu:

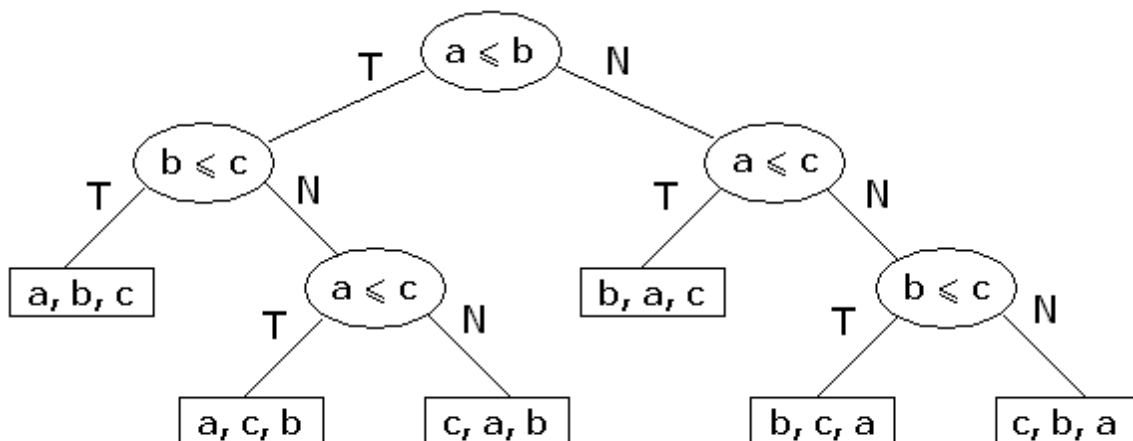
- Bubble-, Selection-, Insertion-sort,
- Mergesort,
- Heapsort,
- Quicksort,
- i wiele innych algorytmów sortowania.

W tym modelu nieistotna jest organizacja pamięci przechowującej sortowane elementy,

- w szczególności operacje przesuwania i kopiowania.

## Definicja.

- Drzewo decyzyjne sortujące to drzewo binarne, w którym:
- każdy węzeł wewnętrzny zawiera porównanie  $? a_i < a_j$  dla pewnych elementów sortowanego ciągu;
- obliczenie rozpoczyna się w korzeniu;
- jeśli wynik porównania jest "tak" to sterowanie przechodzi do lewego poddrzewa, a jeśli nie, to do prawego;
- na końcu każdej ścieżki jest liść zawierający wynik;
- wynik jest permutacją, w jakiej należy ustawić elementy, by kolejność była rosnąca.



Drzewo decyzyjne sortujące trzy elementy.

Jaki to algorytm sortowania?

Zauważ: każdy wynik-liść jest osiągalny.

## Definicja

Pesymistyczna złożoność obliczeniowa drzewa decyzyjnego to jego wysokość, czyli liczba porównań na najdłuższej ścieżce od korzenia do liścia.

**Fakt 1:** Wysokość drzewa binarnego o  $m$  liściach wynosi co najmniej  $\lg m$ .

bo: jeśli drzewo binarne ma wysokość  $h$  to może mieć co najwyżej  $2^h$  liści

**Fakt 2:** Każde drzewo decyzyjne sortujące  $n$  elementów ma co najmniej  $n!$  liści

bo: tyle jest możliwych wyników – permutacji.

**Zatem:**

Drzewo decyzyjne sortujące  $n$  elementów ma wysokość co najmniej  $\lg(n!)$ .

**Wniosek 1:**

Każdy algorytm sortujący metodą porównań w pesymistycznym przypadku wykonuje co najmniej  $\lg(n!)$  porównań.

**Fakt 3:** (ze wzoru Stirlinga na  $n!$ )  
 $\lg(n!) \approx n \lg n - 1.45n$

**Uwaga:**

Drzewo sortujące może mieć więcej niż  $n!$  liści, ale wśród nich dokładnie  $n!$  liści jest osiągalnych jako wynik algorytmu, bo:

- działanie algorytmu (ścieżka do liścia) zależy tylko od wyników porównań, czyli permutacji, a nie od konkretnych wartości kluczy,
- algorytm jest deterministyczny.

**Ćwiczenie pożyteczne:** rozrysuj drzewo Bubblesort dla  $n=3$  - ile liści nieosiągalnych zawiera?

### **Złożoność średnia w modelu drzew decyzyjnych:**

przy założeniu że wszystkie permutacje są jednakowo prawdopodobne:

jest średnią arytmetyczną długości wszystkich ścieżek do liści osiągalnych.

### **Fakt 4 (kombinatoryczny):**

Dla każdego drzewa binarnego posiadającego  $m$  liści, suma długości ścieżek do liści w tym drzewie wynosi co najmniej  $m \lg m$ .

*Dowód dość łatwy pomijamy.*

Zatem średnia długość ścieżki w drzewie decyzyjnym sortującym  $n$  elementów wynosi co najmniej

$$n! \lg(n!) / n! = \lg(n!).$$

Uwaga: to jest niepoprawnie uproszczona analiza, pomija kwestię liści nieosiągalnych w drzewie decyzyjnym.

### **Wniosek 2:**

Każdy algorytm sortujący metodą porównań w przypadku średnim wykonuje co najmniej  $\approx n \lg n - 1.45n$  porównań.

A zatem dolne ograniczenie na złożoność średnią sortowania jest identyczne jak dla złożoności pesymistycznej.

**Podsumowanie szybkich metod sortowania,**  
pod względem liczby wykonywanych porównań:

- Mergesort jest optymalny (z dokładnością do składnika wielkości  $O(n)$ ) w przypadku pesymistycznym i średnim;
- Quicksort w przypadku średnim jest ok. 1.4 razy gorszy niż dolne ograniczenie; w przypadku pesymistycznym ma rząd złożoności taki jak proste algorytmy sortowania:  $\Theta(n^2)$ ;
- Heapsort jest w przypadku pesymistycznym i średnim około 2 razy gorszy niż dolne ograniczenie; wariant Floyd'a w przypadku średnim jest optymalny z dokładnością do dość dużego składnika  $O(n)$ .

## Sortowanie zewnętrzne

to sortowanie plików, czyli zbiorów danych nie mieszczących się w pamięci operacyjnej.

Założenie: ciąg danych do sortowania  $a_1, \dots, a_n$ , ma strukturę pliku, tzn.

- aby odczytać  $a_k$  należy bezpośrednio przedtem odczytać  $a_1, \dots, a_{k-1}$ ,
- aby zapisać nową wartość  $a_k$  należy zapisać bezpośrednio przedtem  $a_1, \dots, a_{k-1}$ .

Uwaga: mimo że te założenia przypominają rzadko dziś używane taśmy magnetyczne, to praktycznie stosują się również do sortowania dużych zbiorów dyskowych.

### Metoda zrównoważona (von Neumann)

- $p_1, p_2, p_3, p_4$  – pliki fizycznie na ogół odrębne urządzenia;
- dane wejściowe:  $n$  rekordów o kluczach  $a_1, \dots, a_n$ , na pliku  $p_4$ ;
- pamięć wewnętrzna (RAM) mieści  $m$  rekordów.

**Krok 1:** (sortowanie wstępne i rozdzielanie).

- wczytuj po  $m$  kolejnych rekordów do pamięci operacyjnej,
- sortuj i zapisuj posortowane ciągi długości  $m$  na pliki  $p_1$  i  $p_2$  na przemian.

**Krok 2:** (scalanie i rozdzielanie).

- scalaj po jednym posortowanym ciągu z  $p_1$  i  $p_2$  i zapisuj na przemian na pliki  $p_3$  i  $p_4$ ,
- długość posortowanych ciągów podwaja się.

**Iteracja:** Powtarzaj krok 2 zamieniając rolami p1, p2 z p3, p4 aż do osiągnięcia sytuacji gdy pozostaje jeden plik posortowany zawierający cały ciąg.

Algorytm scalania posortowanych ciągów:

- taki sam jak dla tablic posortowanych
- (zamiast dwóch tablic wejściowych są dwa ciągi posortowane: jeden z jednego, drugi z drugiego pliku).

**Przykład:**

po kroku 1 mamy 16 ciągów rosnących długości m rozdzielonych na p1 i p2.

<k> oznacza k-ty w kolejności utworzenia ciąg:

p1: <1> <3> <5> <7> <9> <11> <13> <15>

p2: <2> <4> <6> <8> <10> <12> <14> <16>

po kroku 2:

p3: (<1>\*<2>) (<5>\*<6>) (<9>\*<10>) (<13>\*<14>)

p4: (<3>\*<4>) (<7>\*<8>) (<11>\*<12>) (<15>\*<16>)

(<k>\*<l>) oznacza ciąg powstały ze scalenia posortowanych ciągów <k> oraz <l>

następna iteracja kroku 2 daje:

p1: (<1>\*<2>\*<3>\*<4>) (<9>\*<10>\*<11>\*<12>)

p2: (<5>\*<6>\*<7>\*<8>) (<13>\*<14>\*<15>\*<16>)

(ciągi posortowane są już 4m-elementowe)

następna:

p3: (<1>\*<2>\*<3>\*<4>\*<5>\*<6>\*<7>\*<8>)

p4: (<9>\*<10>\*<11>\*<12>\*<13>\*<14>\*<15>\*<16>)

(po jednym ciągu długości  $n/2$  na każdym pliku)

Ostatnia iteracja daje cały ciąg posortowany (scalonych 16 ciągów początkowych) na p1.

Jeśli  $n$  nie jest potęgą dwójki to pojawiają się ciągi "bez pary" – przepisuje się je.

### **Złożoność mierzy się liczbą:**

- faz (czyli iteracji kroku 2) - jest ich  $\lceil \lg(n/m) \rceil$  ;
- odczytów i zapisów rekordów na pliki:  
w każdej fazie jest  $n$  odczytów,  $n$  zapisów, oraz nie więcej niż  $n$  porównań kluczy, w sumie  $\Theta(n \lg(n/m))$  operacji;
- dodatkowo faza 0: sortowanie wstępne fragmentów  $m$ -elementowych:  
zakładając sortowanie w czasie  $O(m \log m)$ ,  
 $\Theta(n/m * m \log m) = \Theta(n \log m)$  porównań
- liczba porównań razem we wszystkich fazach:  
*zakładając ten sam współczynnik w notacji  $\Theta$  w fazie 0 i następnych*  
 $\Theta(n \lg(n/m)) + \Theta(n \log m) = \Theta(n \log n)$

### **Uwagi:**

- liczba porównań na ogół nie zależy od liczby faz,
- "przewinięcie" plików na początek kosztuje znaczący ułamek czasu, zatem praktycznie czas całości zależy istotnie od liczby faz, a ta maleje ze wzrostem pamięci RAM (parametr  $m$ ).

## Scalanie wielokierunkowe

Mając  $2k$  plików,  $k > 2$ , można wykonywać scalanie równocześnie nie dwóch lecz  $k$  ciągów (scalanie  $k$ -kierunkowe).

Liczba faz maleje:  $\log_k(n/m)$ .

Zakładając  $m=1$  oraz  $k-1$  porównań potrzebnych do wyłonienia kolejnego elementu podczas scalania, liczba porównań wynosi

$$\Theta(k \cdot n \cdot \log_k n)$$

Jeśli  $k$  jest duże, to można kolejne elementy przy scalaniu wyłaniać utrzymując kopiec (kolejkę priorytetową) długości  $k$  zawierający klucz aktualnie czytany z każdego wejściowego pliku.

Wówczas kolejny element zapisywany na wyjście otrzymywany jest w wyniku operacji ExtractMin, a "doczytywany" z tego samego pliku wejściowego element na miejsce usuniętego wstawiany jest operacją Insert (można obie operacje połączyć w jedną "ReplaceMin").

Wówczas liczba porównań wynosi

$$\Theta(n \cdot \lg k \cdot \log_k(n/m))$$

plus koszt początkowego tworzenia kocka w każdej fazie.

## Sortowanie polifazowe

Dysponując  $k+1$  plikami można rozdzielić początkowe ciągi nie równomiernie, jak w metodzie von Neumanna, ale tak, aby potem we wszystkich etapach móc stosować scalanie  $k$ -kierunkowe.

Dla  $k=2$  wstępne liczby ciągów to liczby Fibonacciego.

Przykład: liczba ciągów  $n/m = 13$ ,  
rozdzielone na 2 pliki po 8 i 5 ciągów:

p1: <1> <2> <3> <4> <5> <6> <7> <8>  
p2: <9> <10> <11> <12> <13>  
p3: pusty

Scalanie podciągów parami, z dwóch niepustych plików  
na aktualnie pusty plik, aż jeden ze scalanych plików  
będzie pusty.

Kolejne fazy dają rezultat następujący:

p1: <6> <7> <8>  
p2: pusty  
p3: <1>\*<9> <2>\*<10> <3>\*<11> <4>\*<12> <5>\*<13>

p1: pusty  
p2: <6>\*<1>\*<9> <7>\*<2>\*<10> <8>\*<3>\*<11>  
p3: <4>\*<12> <5>\*<13>

p1: <4>\*<12>\*<6>\*<1>\*<9> <5>\*<13>\*<7>\*<2>\*<10>  
p2: <8>\*<3>\*<11>  
p3: pusty

p1: <5>\*<13>\*<7>\*<2>\*<10>  
p2: pusty  
p3: <8>\*<3>\*<11>\*<4>\*<12>\*<6>\*<1>\*<9>

p1: pusty  
p2: <5>\*<13>\*<7>\*<2>\*<10>\*<8>\*<3>\*<11>\*<4>\*<12>\*<6>\*<1>\*<9> // cały ciąg  
p3: pusty

Metodę można uogólnić na dowolną wartość parametru  
 $k \geq 3$  (uogólnione liczby Fibonacciego).